L33 — Binary Trees: Terminology and Representation

Unit: 3 | **Chapter:** Trees | **BT Level:** BT3 | **CO:** CO2

Learning Objectives

- Define binary tree and its structural properties
- Distinguish full, complete, and perfect binary trees
- Represent binary trees using linked nodes and arrays
- Calculate height, number of nodes, and other tree properties

---

1. Tree Fundamentals

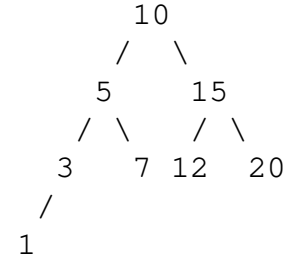
A **tree** is a hierarchical data structure consisting of nodes connected by edges, with no cycles.

Term	Definition
Root	The topmost node (no parent)
Leaf	Node with no children
Internal node	Node with at least one child
Parent	Node directly above another node
Child	Node directly below another node
Sibling	Nodes with the same parent
Height	Length of longest path from root to leaf
Depth	Distance from root to a given node
Level	Set of nodes at the same depth
Subtree	A node and all its descendants

---

2. Binary Tree

A **binary tree** is a tree where each node has **at most two children** — a left child and a right child.

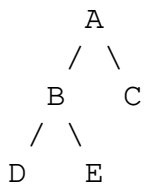


---

3. Types of Binary Trees

3.1 Full Binary Tree

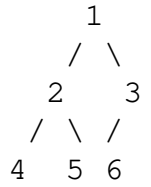
Every node has **0 or 2 children** (never 1).



Property: If n internal nodes → $n+1$ leaves.

3.2 Complete Binary Tree

All levels are **fully filled** except possibly the last, which is filled from **left to right**.



Property: n nodes → height = $\lfloor \log_2 n \rfloor$

3.3 Perfect Binary Tree

All internal nodes have exactly 2 children and all leaves are at the **same level**.

Properties:

- $n = 2^{(h+1)} - 1$ nodes for height h
- 2^h leaves at height h

3.4 Balanced Binary Tree

Height is O(log n) — no path is significantly longer than another.

Example: AVL tree, Red-Black tree.

3.5 Degenerate (Skewed) Tree

Every parent has only one child — effectively a linked list.

Height: O(n) — worst case for most tree operations.

---

4. Binary Tree Representation

4.1 Linked Node Representation

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# Build the tree:   10
#                  /  \
#                 5    15
#                / \
#               3   7

root = TreeNode(10)
root.left = TreeNode(5)
root.right = TreeNode(15)
root.left.left = TreeNode(3)
root.left.right = TreeNode(7)
```

Memory layout for each node:

```
+------+-------+-------+
| data | left* | right*|
+------+-------+-------+
  4B     8B      8B     = 20 bytes per node (64-bit system)
```

4.2 Array Representation (for Complete/Perfect Trees)

Index the nodes level by level, left to right (1-indexed):

```
Level 0:    1          → index 1
Level 1:   2 3         → indices 2, 3
Level 2:  4 5 6 7      → indices 4, 5, 6, 7

Parent of index i: i // 2
Left child of i:   2 * i
Right child of i:  2 * i + 1
```

```
class ArrayBinaryTree:
    def __init__(self, capacity):
        self.data = [None] * (capacity + 1)  # 1-indexed
        self.size = 0

    def insert(self, value):
        """Insert using level-order (next available position)."""
        self.size += 1
        self.data[self.size] = value

    def left_child(self, i):
        idx = 2 * i
        return self.data[idx] if idx <= self.size else None

    def right_child(self, i):
        idx = 2 * i + 1
```

```
        return self.data[idx] if idx <= self.size else None

    def parent(self, i):
        return self.data[i // 2] if i > 1 else None

    def is_leaf(self, i):
        return 2 * i > self.size
```

---

5. Binary Tree Properties

Property	Formula
Max nodes at level k	2^k
Max nodes in tree of height h	$2^{(h+1)} - 1$
Min height for n nodes	$\lceil \log_2(n+1) \rceil - 1$
Height of complete tree (n nodes)	$\lfloor \log_2 n \rfloor$

---

6. Tree Measurement Functions

```
def height(node):
    """Height of subtree rooted at node. Empty tree height = -1."""
    if node is None:
        return -1
    return 1 + max(height(node.left), height(node.right))


def count_nodes(node):
    """Total number of nodes in subtree."""
    if node is None:
        return 0
    return 1 + count_nodes(node.left) + count_nodes(node.right)


def count_leaves(node):
    """Number of leaf nodes."""
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return 1
    return count_leaves(node.left) + count_leaves(node.right)


def is_full_binary_tree(node):
    """Check if tree is a full binary tree."""
    if node is None:
        return True
    if node.left is None and node.right is None:
        return True   # Leaf
    if node.left is not None and node.right is not None:
```

```
        return is_full_binary_tree(node.left) and is_full_binary_tree(n
    return False   # Exactly one child — not full
```

---

7. Summary of Traversals (Preview)

Binary trees support three standard DFS traversals:

Traversal	Order	Use Case
Inorder	Left → Root → Right	BST sorted order
Preorder	Root → Left → Right	Copy/serialize tree
Postorder	Left → Right → Root	Delete/evaluate tree
Level-order	Level by level (BFS)	Shortest path

Summary

- Binary tree: each node has $\leq$ 2 children (left/right).
- Types: full, complete, perfect, balanced, degenerate/skewed.
- Linked representation: natural for insertions/deletions.
- Array representation: efficient for complete trees (heaps).
- Height determines complexity of operations: O(h) per operation.

References

- Cormen et al., *Introduction to Algorithms*, Ch. 12 (MIT Press, 2022)
- Skiena, *The Algorithm Design Manual*, Ch. 3.4 (Springer, 2020)